BM25S: Orders of magnitude faster lexical search via eager sparse scoring

Xing Han Lù
McGill University and Mila Quebec AI Institute
xing.han.lu@mail.mcgill.ca

arXiv:2407.03618v1 [cs.IR] 4 Jul 2024

Abstract

We introduce BM25S, an efficient Python-based implementation of BM25 that only depends on Numpy[1] and Scipy[2]. BM25S achieves up to a 500x speedup compared to the most popular Python-based framework by eagerly computing BM25 scores during indexing and storing them into sparse matrices. It also achieves considerable speedups compared to highly optimized Java-based implementations, which are used by popular commercial products. Finally, BM25S reproduces the exact implementation of five BM25 variants based on Kamphuis et al. (2020) by extending eager scoring to non-sparse variants using a novel score shifting method. The code can be found at https://github.com/xhluca/bm25s

1 Background

Sparse lexical search algorithms, such as the BM25 family (Robertson et al., 1995) remain widely used as they do not need to be trained, can be applied to multiple languages, and are generally faster, especially when using highly efficient Java-based implementations. Those Java implementations, usually based on Lucene[3], are accessible inside Python via the Pyserini reproducibility toolkit (Lin et al., 2021), and through HTTP by using the Elasticsearch web client[4]. The Lucene-based libraries are known to be faster than existing Python-based implementations, such as Rank-BM25[5].

This work shows that it is possible to achieve a significant speedup compared to existing Python-based implementations by introducing two improvements: eagerly calculating all possible scores that can be assigned to any future query token when indexing a corpus, and storing those calculations inside sparse matrices to enable faster slicing and summations. The idea of sparse matrices was previously explored in BM25-PT[6], which precomputes BM25 scores using PyTorch and multiplies them with a bag-of-word encoding of the query via sparse matrix multiplication.

This work expands upon the initial idea proposed by the BM25-PT project by significantly simplifying the implementation and introducing a strategy to generalize to other variants of the original BM25. Unlike BM25-pt, BM25S does not rely on PyTorch, and instead uses Scipy's sparse matrix implementation. Whereas BM25-PT multiplies bag-of-words with the document matrix, BM25S instead slices relevant indices and sums across the token-dimension, removing the need of matrix multiplications.

At the implementation level, BM25S also introduces a simple but fast Python-based tokenizer that combines Scikit-Learn's text splitting (Pedregosa et al., 2011), Elastic's stopword list[7], and (optionally) integrates a C-based implementation of the Snowball stemmer (Bouchet-Valat, 2014). This achieves a better performance compared to subword tokenizers (Kudo and Richardson, 2018) used by BM25-PT. Finally, it implements top-k retrieval using an average $O(n)$ time complexity when selecting the K most relevant documents from a set of n scores associated with each document.

2 Implementation

The implementation described below follows the study by Kamphuis et al. (2020).

Calculation of BM25 Many variants of BM25 exist, which could lead to significant confusion about the exact scoring method used in a given implementation (Kamphuis et al., 2020). By default, we use the scoring method proposed by Lucene. Thus, for a given query Q (tokenized into $q_1, \ldots, q_{|Q|}$) and document D from collection C,

[1] https://numpy.org/
[2] https://scipy.org/
[3] https://lucene.apache.org/
[4] https://www.elastic.co/elasticsearch
[5] https://github.com/dorianbrown/rank_bm25
[6] https://github.com/jxmorris12/bm25_pt
[7] https://www.elastic.co/guide/en/elasticsearch/guide/current/stopwords.html

we compute the following score[8]:

$$B(Q, D) = \sum_{i=1}^{|Q|} S(q_i, D)$$
$$= \sum_{i=1}^{|Q|} \text{IDF}(q_i, C) \frac{\text{TF}(q_i, D)}{\mathcal{D}}$$

where $\mathcal{D} = \text{TF}(t, D) + k_1 \left(1 - b + b\frac{|D|}{L_{avg}}\right)$, L_{avg} is the average length of documents in corpus C (calculated in number of tokens), $\text{TF}(q_i, D)$ is the term frequency of token q_i within the set of tokens in D. The *IDF* is the inverse document frequency, which is calculated as:

$$\text{IDF}(q_i, C) = \ln\left(\frac{|C| - \text{DF}(q_i, C) + 0.5}{\text{DF}(q_i, C) + 0.5} + 1\right)$$

Where document frequency $\text{DF}(q_i, C)$ is the number of documents in C containing q_i. Although $B(Q, D)$ depends on the query, which is only given during retrieval, we show below how to reformulate the equation to eagerly calculate the TF and IDF during indexing.

Eager index-time scoring Let's now consider all tokens in a vocabulary V, denoted by $t \in V$. We can reformulate $S(t, D)$ as:

$$S(t, D) = \text{TF}(t, D) \cdot \text{IDF}(t, C) \frac{1}{\mathcal{D}}$$

When t is a token that is not present in document D, then $\text{TF}(t, D) = 0$, leading to $S(t, D) = 0$ as well. This means that, for most tokens in vocabulary V, we can simply set the relevance score to 0, and only compute values for t that are actually in the document D. This calculation can be done during the indexing process, thus avoiding the need to compute $S(q_i, D)$ at query time, apart from straightforward summations.

Assigning Query Scores Given our sparse matrix of shape $|V| \times |C|$, we can use the query tokens to select relevant rows, leaving us a matrix of shape $|Q| \times |C|$, which we can then sum across the column dimension, resulting in a single $|C|$-dimension vector (representing the score of the score of each document for the query).

Efficient Matrix Sparsity We implement a sparse matrix in *Compressed Sparse Column* (CSC) format (`scipy.sparse.csc_matrix`)[9], which provides an efficient conversion between the coordinate and CSC format. Since we slice and sum alongside the column dimension, this implementation is the optimal choice among sparse matrix implementations. In practice, we replicate the sparse operations directly using Numpy array.

Tokenization To split the text, we use the same Regular Expression pattern used by Scikit-Learn (Pedregosa et al., 2011) for their own tokenizers, which is `r"(?u)\b\w\w+\b"`. This pattern conveniently parses words in UTF-8 (allowing coverage of various languages), with `\b` handling word boundaries. Then, if stemming is desired, we can stem all words in the vocabulary, which can be used to look up the stemmed version of each word in the collection. Finally, we build a dictionary mapping each unique (stemmed) word to an integer index, which we use to convert the tokens into their corresponding index, thus significantly reducing memory usage and allowing them to be used to slice Scipy matrices and Numpy arrays.

Top-k selection Upon computing scores for all documents in a collection, we can complete the search process by selecting the top-k most relevant elements. A naive approach to this would be to sort the score vector and select the last k elements; instead, we take the partition of the array, selecting only the last k documents (unordered). Using an algorithm such as Quickselect (Hoare, 1961), we can accomplish this in an average time complexity of $O(n)$ for n documents in the collection, whereas sorting requires $O(n \log n)$. If the user wishes to receive the top-k results in order, sorting the partitioned documents would take an additional $O(k \log k)$, which is a negligible increase in time complexity assuming $k \ll n$. In practice, `BM25S` allows the use of two implementations: one based in `numpy`, which leverages `np.argpartition`, and another in `jax`, which relies on XLA's top-k implementation. Numpy's `argpartition` uses[10] the introspective selection algorithm (Musser, 1997), which modifies the quickselect algorithm to ensure that the worst-case performance remains in $O(n)$. Although this guarantees optimal time complexity, we observe that JAX's implementation achieves better performance in practice.

[8] We follow notations by Kamphuis et al. (2020)

[9] https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csc_matrix.html

[10] https://numpy.org/doc/stable/reference/generated/numpy.argpartition.html

Dataset	BM25S	ES	PT	Rank
ArguAna	**573.91**	13.67	**110.51**	2.00
Climate-FEVER	13.09	4.02	OOM	0.03
CQADupstack	**170.91**	13.38	OOM	0.77
DBPedia	13.44	10.68	OOM	0.11
FEVER	20.19	7.45	OOM	0.06
FiQA	**507.03**	16.96	20.52	4.46
HotpotQA	20.88	7.11	OOM	0.04
MSMARCO	12.20	11.88	OOM	0.07
NFCorpus	**1196.16**	45.84	**256.67**	**224.66**
NQ	41.85	12.16	OOM	0.10
Quora	**183.53**	21.80	6.49	1.18
SCIDOCS	**767.05**	17.93	41.34	9.01
SciFact	**952.92**	20.81	**184.30**	47.60
TREC-COVID	**85.64**	7.34	3.73	1.48
Touche-2020	**60.59**	13.53	OOM	1.10

Table 1: To calculate the throughput, we calculate the number of *queries per second (QPS)* that each model can process for each task in the public section of the BEIR leaderboard; instances achieve over 50 QPS are shown in **bold**. We compare `BM25S`, BM25-PT (PT), Elasticsearch (ES) and Rank-BM25 (Rank). OOM indicates failure due to out-of-memory issues.

Multi-threading We implement optional multi-threading capabilities through pooled executors[11] to achieve further speed-up during retrieval.

Alternative BM25 implementations Above, we describe how to implement `BM25S` for one variant of BM25 (namely, Lucene). However, we can easily extend the `BM25S` method to many variants of BM25; the sparsity can be directly applied to Robertson's original design (Robertson et al., 1995), ATIRE (Trotman et al., 2014), and Lucene. For other models, a modification of the scoring described above is needed.

2.1 Extending sparsity via non-occurrence adjustments

For BM25L (Lv and Zhai, 2011), BM25+ (Lv and Zhai, 2011) and $\text{TF}_{l\circ\delta\circ p}\times\text{IDF}$ (Rousseau and Vazirgiannis, 2013), we notice that when $TF(t, D) = 0$, the value of $S(t, D)$ will not be zero; we denote this value as a scalar[12] $S^\theta(t)$, which represents the score of t when it does not occur in document D.

Clearly, constructing a $|V| \times |C|$ dense matrix would use up too much memory[13]. Instead, we can still achieve sparsity by subtracting $S^\theta(t)$ from each token t and document D in the score matrix (since most tokens t in the vocabulary will not be present in any given document D, their value in the score matrix will be 0). Then, during retrieval, we can simply compute $S^\theta(q_i)$ for each query $q_i \in Q$, and sum it up to get a single scalar that we can add to the final score (which would not affect the rank).

More formally, for an empty document $\emptyset$, we define $S^\theta(t) = S(t, \emptyset)$ as the nonoccurrence score for token t. Then, the differential score $S^\Delta(t, D)$ is defined as:

$$S^\Delta(t, D) = S(t, D) - S^\theta(t)$$

Then, we reformulate the BM25 (B) score as:

$$\begin{aligned} B(Q, D) &= \sum_{i=1}^{|Q|} S(q_i, D) \\ &= \sum_{i=1}^{|Q|} \Big(S(q_i, D) - S^\theta(q_i) + S^\theta(q_i) \Big) \\ &= \sum_{i=1}^{|Q|} \Big(S^\Delta(q_i, D) + S^\theta(q_i) \Big) \\ &= \sum_{i=1}^{|Q|} S^\Delta(q_i, D) + \sum_{i=1}^{|Q|} S^\theta(q_i) \end{aligned}$$

where $\sum_{i=1}^{|Q|} S^\Delta(q_i, D)$ can be efficiently computed using the differential sparse score matrix (the same way as ATIRE, Lucene and Robertson) in `scipy`. Also, $\sum_{i=1}^{|Q|} S^\theta(q_i)$ only needs to be computed once for the query Q, and can be subsequently applied to every retrieved document to obtain the exact scores.

3 Benchmarks

Throughput For benchmarking, we use the publicly available datasets from the BEIR benchmark (Thakur et al., 2021). Results in Table 1 show that `BM25S` is substantially faster than Rank-BM25, as it achieves over 100x higher throughput in 10 out of the 14 datasets; in one instance, it achieves a 500x speedup. Further details can be found in Appendix A.

Impact of Tokenization We further examine the impact of tokenization on each model in Table 2 by comparing `BM25S` Lucene with $k_1 = 1.5$ and $b = 0.75$ (1) without stemming, (2) without stop words, and (3) with neither, and (4) with both. On average, adding a Stemmer improves the score on average, wheareas the stopwords have minimal impact. However, on individual cases, the stopwords can have a bigger impact, such as in the case of Trec-COVID (TC) and ArguAna (AG).

[11] Using `concurrent.futures.ThreadPoolExecutor`

[12] We note that it is not an $|D|$-dimensional array since it does not depend on D, apart from the document frequency of t, which can be represented with a $|V|$-dimensional array.

[13] For example, we would need 1.6TB of RAM to store a dense matrix of 2M documents with 200K words in the vocabulary.

Stop	Stem	Avg.	AG	CD	CF	DB	FQ	FV	HP	MS	NF	NQ	QR	SD	SF	TC	WT
Eng.	None	38.4	48.3	29.4	13.1	27.0	23.3	48.2	56.3	21.2	30.6	27.3	74.8	15.4	66.2	59.5	35.8
Eng.	Snow.	39.7	49.3	29.9	13.6	29.9	25.1	48.1	56.9	21.9	32.1	28.5	80.4	15.8	68.7	62.3	33.1
None	None	38.3	46.8	29.6	13.6	26.6	23.2	48.8	56.9	21.1	30.6	27.8	74.2	15.2	66.1	58.3	35.9
None	Snow.	39.6	47.7	30.2	13.9	29.5	25.1	48.7	57.5	21.7	32.0	29.1	79.7	15.6	68.5	61.6	33.4

Table 2: NDCG@10 results of different tokenization schemes (including and excluding stopwords and the Snowball stemmer) on all BEIR dataset (Appendix A provides a list of datasets). We notice that including both stopwords and stemming modestly improves the performance of the BM25 algorithm.

k_1	b	Variant	Avg.	AG	CD	CF	DB	FQ	FV	HP	MS	NF	NQ	QR	SD	SF	TC	WT
1.5	0.75	BM25PT	–	44.9	–	–	–	22.5	–	–	–	31.9	–	75.1	14.7	67.8	58.0	–
1.5	0.75	PSRN	40.0*	48.4	–	14.2	30.0	25.3	50.0	57.6	22.1	32.6	28.6	80.6	15.6	68.8	63.4	33.5
1.5	0.75	R-BM25	39.6	49.5	29.6	13.6	29.9	25.3	49.3	58.1	21.1	32.1	28.5	80.3	15.8	68.5	60.1	32.9
1.5	0.75	Elastic	42.0	47.7	29.8	17.8	31.1	25.3	62.0	58.6	22.1	34.4	31.6	80.6	16.3	69.0	68.0	35.4
1.5	0.75	Lucene	39.7	49.3	29.9	13.6	29.9	25.1	48.1	56.9	21.9	32.1	28.5	80.4	15.8	68.7	62.3	33.1
0.9	0.4	Lucene	41.1	40.8	28.2	16.2	31.9	23.8	63.8	62.9	22.8	31.8	30.5	78.7	15.0	67.6	58.9	44.2
1.2	0.75	Lucene	39.9	48.7	30.1	13.7	30.3	25.3	50.3	58.5	22.6	31.8	29.1	80.5	15.6	68.0	61.0	33.2
1.2	0.75	ATIRE	39.9	48.7	30.1	13.7	30.3	25.3	50.3	58.5	22.6	31.8	29.1	80.5	15.6	68.1	61.0	33.2
1.2	0.75	BM25+	39.9	48.7	30.1	13.7	30.3	25.3	50.3	58.5	22.6	31.8	29.1	80.5	15.6	68.1	61.0	33.2
1.2	0.75	BM25L	39.5	49.6	29.8	13.5	29.4	25.0	46.6	55.9	21.4	32.2	28.1	80.3	15.8	68.7	62.9	33.0
1.2	0.75	Robertson	39.9	49.2	29.9	13.7	30.3	25.4	50.3	58.5	22.6	31.9	29.2	80.4	15.5	68.3	59.0	33.8

Table 3: Comparison of different variants and parameters on all BEIR dataset (Appendix A provides a list of datasets). Following the recommended range of $k_1 \in [1.2, 2]$ by Schütze et al. (2008), we try both $k_1 = 1.5$ and $k_1 = 1.2$ with $b = 0.75$. Additionally, we use $k_1 = 0.9$ and $b = 0.4$ following the parameters recommend in BEIR. We additionally benchmark five of the BM25 variants described in Kamphuis et al. (2020). *note that Pyserini's average results are estimated, as the experiments for CQADupStack (CD) did not terminate due to OOM errors.

Comparing model variants In Table 3, we compare many implementation variants, including commercial (Elasticsearch) offerings and reproducibility toolkits (Pyserini). We notice that most implementations achieve an average be between 39.7 and 40, with the exception of Elastic which achieves a marginally higher score. The variance can be attributed to the difference in the tokenization scheme; notably, the subword tokenizer used in BM25-PT likely lead to the difference in the results, considering the implementation is a hybrid between ATIRE and Lucene, both of which achieve better results with a word-level tokenizer. Moreover, although Elasticsearch is built on top of Lucene, it remains an independent commercial product, and the documentations[14] do not clearly describe how they are splitting the text[15], and whether they incorporate additional processing beyond the access to a Snowball stemmer and the removal of stopwords.

[14] https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html

[15] https://www.elastic.co/guide/en/elasticsearch/reference/current/split-processor.html

4 Conclusion

We provide a novel method for calculating BM25 scores, `BM25S`, which also offers fast tokenization out-of-the-box and efficient top-k selection during querying, minimizes dependencies and makes it usable directly inside Python. As a result, `BM25S` naturally complements previous implementations: BM25-pt can be used with PyTorch, Rank-BM25 allows changing parameters $k1$ during inference, and Pyserini provides a large collection of both sparse and dense retrieval algorithm, making it the best framework for reproducible retrieval research. On the other hand, `BM25S` remains focused on sparse and mathematically accurate implementations of BM25 that leverage the eager sparse scoring methods, with optional Python dependencies like PyStemmer for stemming and Jax for top-k selection. By minimizing dependencies, `BM25S` becomes a good choice in scenarios where storage might be limited (e.g. for edge deployments) and can be used in the browser via WebAssembly frameworks like Pyodide[16] and Pyscript[17]. We believe our fast and accurate implementation will

[16] https://pyodide.org

[17] https://pyscript.net/

make lexical search more accessible to a broader audience.

Limitations

A customized Python-based tokenizer (also known as analyzer) was created for `BM25S`, which allows the use of stemmer and stopwords. By focusing on a readable, extensible and fast implementation, it may not achieve the highest possible performance. When reporting benchmarks results in research papers, it is worth considering different lexical search implementations in addition to `BM25S`.

Additionally, in order to ensure reproducibility and accessibility, our experiments are all performed on free and readily available hardware (Appendix A). As a result, experiments that are less memory efficient terminated with OOM errors.

Acknowledgements

The author thanks Andreas Madsen and Marius Mosbach for helpful discussions.

References

Alexander Bondarenko, Matthias Hagen, Martin Potthast, Henning Wachsmuth, Meriem Beloucif, Chris Biemann, Alexander Panchenko, and Benno Stein. 2020. Touché: First shared task on argument retrieval. In *Advances in Information Retrieval: 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, April 14–17, 2020, Proceedings, Part II 42*, pages 517–523. Springer.

Milan Bouchet-Valat. 2014. Snowball stemmers based on the c libstemmer utf-8 library.

Daniel Fernando Campos, Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, Li Deng, and Bhaskar Mitra. 2016. Ms marco: A human generated machine reading comprehension dataset. *ArXiv*, abs/1611.09268.

Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. 2020. Specter: Document-level representation learning using citation-informed transformers. *ArXiv*, abs/2004.07180.

Thomas Diggelmann, Jordan L. Boyd-Graber, Jannis Bulian, Massimiliano Ciaramita, and Markus Leippold. 2020. Climate-fever: A dataset for verification of real-world climate claims. *ArXiv*, abs/2012.00614.

Faegheh Hasibi, Fedor Nikolaev, Chenyan Xiong, Krisztian Balog, Svein Erik Bratsberg, Alexander Kotov, and Jamie Callan. 2017. Dbpedia-entity v2: A test collection for entity search. *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

C. A. R. Hoare. 1961. Algorithm 65: find. *Commun. ACM*, 4(7):321–322.

Chris Kamphuis, Arjen P De Vries, Leonid Boytsov, and Jimmy Lin. 2020. Which bm25 do you mean? a large-scale reproducibility study of scoring variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, April 14–17, 2020, Proceedings, Part II 42*, pages 28–34. Springer.

Taku Kudo and John Richardson. 2018. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Conference on Empirical Methods in Natural Language Processing*.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Jimmy J. Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, Rodrigo Nogueira, and David R. Cheriton. 2021. Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations. *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

Yuanhua Lv and ChengXiang Zhai. 2011. Adaptive term frequency normalization for bm25. In *International Conference on Information and Knowledge Management*.

Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. 2018. Www'18 open challenge: Financial opinion mining and question answering. *Companion Proceedings of the The Web Conference 2018*.

David R Musser. 1997. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Gilles Louppe, Peter Prettenhofer, Ron Weiss, Ron J. Weiss, J. Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine learning in python. *ArXiv*, abs/1201.0490.

Kirk Roberts, Tasmeer Alam, Steven Bedrick, Dina Demner-Fushman, Kyle Lo, Ian Soboroff, Ellen M. Voorhees, Lucy Lu Wang, and William R. Hersh. 2020. Trec-covid: rationale and structure of an information retrieval shared task for covid-19. *Journal of the American Medical Informatics Association : JAMIA*, 27:1431 – 1436.

Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford, et al. 1995. Okapi at trec-3. *Nist Special Publication Sp*, 109:109.

François Rousseau and Michalis Vazirgiannis. 2013. Composition of tf normalizations: new insights on scoring functions for ad hoc ir. *Proceedings of the 36th international ACM SIGIR conference on Research and development in information retrieval*.

Hinrich Schütze, Christopher D Manning, and Prabhakar Raghavan. 2008. *Introduction to information retrieval*, volume 39. Cambridge University Press Cambridge.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a large-scale dataset for fact extraction and VERification. In *NAACL-HLT*.

Andrew Trotman, Antti Puurula, and Blake Burgess. 2014. Improvements to bm25 and language models examined. *Proceedings of the 19th Australasian Document Computing Symposium*.

Henning Wachsmuth, Martin Trenkmann, Benno Stein, Gregor Engels, and Tsvetomira Palakarska. 2014. A review corpus for argumentation analysis. In *Conference on Intelligent Text Processing and Computational Linguistics*.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. 2020. Fact or fiction: Verifying scientific claims. In *EMNLP*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing*.

A Appendix

Hardware To calculate the queries per second, we run our experiments using a single-threaded approach. In the interest of reproducibility, our experiments can be reproduced on Kaggle's free CPU instances[18], which are equipped with a Intel Xeon CPU @ 2.20GHz and 30GB of RAM. This setup reflects consumer devices, which tend have fewer CPU cores and rarely exceed 32GB of RAM.

BEIR Datasets BEIR (Thakur et al., 2021) contains the following datasets: Arguana (AG; Wachsmuth et al., 2014), Climate-FEVER (CF; Diggelmann et al., 2020), DBpedia-Entity (DB; Hasibi et al., 2017), FEVER (FV; Thorne et al., 2018), FiQA (FQ; Maia et al., 2018), HotpotQA (HP; Yang et al., 2018), MS MARCO (MS; Campos et al., 2016), NQ (NQ; Kwiatkowski et al., 2019), Quora (QR)[19], SciDocs (SD; Cohan et al., 2020), SciFact (SF; Wadden et al., 2020), TREC-COVID (TC; Roberts et al., 2020), Touche-2020 (WT; Bondarenko et al., 2020).

[18] https://www.kaggle.com/

[19] https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs